

Documentazione Sperimentale

Analisi dell'Impatto della Latenza di Rete su MAPPO in Ambiente RWARE

1. Introduzione e Obiettivo

Negli odierni scenari industriali, come la logistica automatizzata e il controllo di flotte di droni, l'efficienza si basa sulla cooperazione in tempo reale di decine di robot. Tuttavia, il passaggio dalle simulazioni ideali al mondo fisico si scontra con un ostacolo inevitabile: i limiti hardware e la congestione del Wi-Fi introducono latenze temporali che sfalsano la percezione della realtà da parte dei sistemi autonomi.

Il presente documento costituisce la documentazione tecnica e scientifica relativa all'esperimento condotto nell'ambito del progetto per il corso di Network Intelligence (Communication Engineering). L'obiettivo centrale dello studio è dimostrare scientificamente come gli algoritmi standard di Multi-Agent Reinforcement Learning (MARL), pur raggiungendo prestazioni ottimali in condizioni di simulazione ideale, subiscano un degrado critico e strutturale in presenza di ritardi di comunicazione — una condizione che caratterizza in modo intrinseco qualsiasi sistema di controllo distribuito reale, in particolare nelle reti mobili e nei sistemi cyber-fisici.

La scelta dell'ambiente RWARE (Robot Warehouse) come banco di prova non è casuale: si tratta di un ambiente cooperativo multi-agente in cui due robot devono coordinare le proprie azioni per consegnare pacchi nelle posizioni corrette. L'ambiente richiede una cooperazione stretta e una pianificazione temporale precisa, rendendolo particolarmente sensibile a qualsiasi perturbazione nel flusso informativo tra agenti e ambiente. L'algoritmo selezionato per lo studio è MAPPO (Multi-Agent Proximal Policy Optimization), una delle architetture Actor-Critic centralizzate più moderne e performanti nel panorama MARL, che sfrutta un Critico centralizzato per stabilizzare l'apprendimento degli attori decentralizzati.

L'ipotesi di ricerca è la seguente: l'iniezione di un ritardo artificiale (lag) di 2 step sulle osservazioni ricevute dagli agenti — simulando la latenza di rete tipica di un sistema di controllo distribuito — compromette in modo misurabile e documentabile sia la velocità di convergenza dell'algoritmo, sia la stabilità a lungo termine delle policy apprese. I risultati sperimentali, come illustrato nel dettaglio nelle sezioni successive, confermano questa ipotesi

in modo inequivocabile, attraverso tre effetti distinti e correlati: inefficienza logistica intrinseca, cecità informativa del Critico, e fragilità strutturale delle policy ottimizzate.

Come sviluppi futuri, il progetto ha inoltre esplorato due possibili strategie di mitigazione: un Critico Lag-Aware (modifica architetturale di `centralV.py` per rendere il Critico consapevole del ritardo) e un modulo di Multi-Head Attention integrato nell'agente ricorrente (`rnn_agent.py`), con l'obiettivo di dotare il sistema di memoria selettiva sulle osservazioni storiche. I risultati preliminari di questi interventi, pur promettenti, non rientrano nell'analisi principale di questo documento e sono proposti come direzione per ricerche successive.

2. Background e Sistema di Riferimento

Per comprendere l'impatto della latenza di rete, è necessario delineare i due pilastri su cui si basa il sistema di riferimento in condizioni ideali: l'ambiente di simulazione logistica e l'algoritmo di apprendimento multi-agente.

2.1 L'Ambiente di Simulazione: Robot Warehouse (RWARE)

Il banco di prova utilizzato è RWARE, un ambiente grafico discretizzato che simula le dinamiche operative di un magazzino logistico automatizzato.

- **Dinamica di Gioco:** Il sistema prevede una flotta di agenti cooperanti (robot) il cui obiettivo è localizzare specifici scaffali, agganciarli, trasportarli verso una linea di consegna prestabilita e infine ricollocare lo scaffale vuoto.
- **Meccanismo di Reward:** Il sistema adotta una struttura di ricompensa estremamente sparsa. Gli agenti non ricevono feedback positivi per i movimenti intermedi, ma ottengono una ricompensa collettiva (pari a +1) esclusivamente nel momento esatto in cui una consegna viene completata con successo.
- **Vincoli fisici:** Gli episodi hanno una durata fissa e rigida di 500 step temporali. Gli agenti devono muoversi nello spazio evitando collisioni o situazioni di stallo (deadlock), che ne vanificherebbero l'efficienza temporale.

2.2 L'Algoritmo di Controllo: MAPPO

La gestione e l'addestramento dei robot sono affidati a MAPPO (Multi-Agent Proximal Policy Optimization), un algoritmo allo stato dell'arte che estende PPO agli scenari multi-agente seguendo il paradigma CTDE (Centralized Training with Decentralized Execution — Addestramento Centralizzato ed Esecuzione Decentralizzata).

- **L'Attore (Decentralized Execution):** In fase di esecuzione, ogni singolo agente è guidato da una rete Attore locale. L'Attore prende decisioni basandosi esclusivamente sulla propria osservazione parziale dell'ambiente (il raggio visivo circostante il robot), garantendo la scalabilità del sistema.
- **Il Critico (Centralized Training):** In fase di addestramento, interviene una seconda rete neurale globale chiamata Critico. Il Critico è un'entità onnisciente che ha accesso allo stato globale dell'intero magazzino (la posizione di tutti i robot e di tutti gli scaffali). Il suo compito è valutare quanto siano buone le azioni intraprese dagli attori e guidare l'aggiornamento dei loro pesi sinaptici tramite il calcolo della value function.

3. Setup Sperimentale

3.1 Ambiente e Algoritmo

Tutti gli esperimenti sono stati condotti utilizzando il framework EPyMARL, una libreria Python open-source progettata per la ricerca in MARL, sulla variante rware-tiny-2ag-v2 dell'ambiente RWARE — la configurazione minimale a 2 agenti, scelta per isolare con precisione il comportamento del sistema riducendo al minimo le variabili confondenti. L'orizzonte di addestramento è stato fissato a 40 milioni di step per garantire l'osservazione completa della convergenza a lungo termine e dei potenziali fenomeni di instabilità tardiva. Tutte le metriche sono state tracciate in tempo reale tramite Weights & Biases (W&B) sul progetto Progetto-MAPPO-RWARE.

3.2 Configurazioni a Confronto

Per garantire la validità scientifica del confronto, le due configurazioni differiscono esclusivamente per un singolo parametro — la presenza o assenza del lag — mantenendo identici tutti gli altri iperparametri. Questa metodologia di isolamento delle variabili consente di attribuire con certezza qualsiasi differenza osservata esclusivamente all'effetto della latenza.

Configurazione 1 — MAPPO_Baseline_NoLag_40M (caso di controllo ideale):

Gli agenti ricevono le osservazioni dell'ambiente in tempo reale, senza alcun ritardo artificiale. Rappresenta il caso ideale in cui la latenza di rete è assente.

```
python src/main.py --config=mappo --env-config=gymma \
  with env_args.time_limit=500 \
  env_args.key="rware:rware-tiny-2ag-v2" \
  t_max=40000000 \
  use_wandb=True \
  wandb_project="Progetto-MAPPO-RWARE" \
  wandb_team="emilianomazzone2000-university-of-catania" \
  wandb_mode="online"
```

Configurazione 2 — MAPPO_Lag2_40M (caso di test con latenza):

Viene iniettato un ritardo artificiale di 2 step sulle osservazioni ricevute dagli agenti, simulando la latenza di rete tipica di un sistema di controllo distribuito. Il parametro chiave che distingue questa configurazione dalla baseline è esclusivamente `env_args.obs_lag=2`.

```
python src/main.py --config=mappo --env-config=gymma \  
  with env_args.time_limit=500 \  
  env_args.key="rware:rware-tiny-2ag-v2" \  
  env_args.obs_lag=2 \  
  t_max=40000000 \  
  use_wandb=True \  
  wandb_project="Progetto-MAPPO-RWARE" \  
  wandb_team="emilianomazzone2000-university-of-catania" \  
  wandb_mode="online"
```

3.3 Descrizione dei Parametri

- **--config=mappo:** seleziona il file di configurazione dell'algoritmo MAPPO (src/config/alg/mappo.yaml).
- **--env-config=gymma:** specifica il wrapper dell'ambiente, compatibile con le librerie Gym-based come RWARE.
- **env_args.time_limit=500:** fissa la durata massima di ogni episodio a 500 step. Entrambe le configurazioni condividono questo limite.
- **env_args.key:** identifica univocamente la variante dell'ambiente: rware-tiny-2ag-v2 è la configurazione minimale con 2 agenti.
- **env_args.obs_lag=2:** parametro critico: introduce un ritardo di 2 step sulle osservazioni. Assente nella baseline, presente nel caso di test. È il singolo parametro che distingue i due esperimenti.
- **t_max=40000000:** numero totale di step di addestramento (40 milioni), scelto per osservare la convergenza completa.
- **use_wandb=True:** abilita il logging in tempo reale su Weights & Biases per il tracciamento delle metriche.
- **wandb_project / wandb_team:** identificano il progetto e il team su W&B per l'organizzazione dei run.
- **wandb_mode="online":** modalità di sincronizzazione continua con il server W&B durante il training.

4. Analisi dei Risultati

4.1 Efficienza e Dinamiche degli Episodi

Un primo elemento di fondamentale importanza emerge dall'osservazione del grafico `test_ep_length_mean`: per entrambe le configurazioni sperimentali, la durata media degli episodi di test rimane costantemente e rigidamente fissata a 500 step per l'intera durata del training, senza alcuna deviazione apprezzabile. Questo dato, apparentemente banale, riveste in realtà un significato diagnostico cruciale per l'interpretazione complessiva dei risultati.

L'assenza di variazione nella lunghezza degli episodi esclude categoricamente che il degrado prestazionale osservato nella configurazione `MAPPO_Lag2_40M` sia riconducibile a terminazioni anticipate degli episodi stessi — fenomeno che si osserverebbe qualora gli agenti incorressero frequentemente in collisioni fatali o stati terminali non desiderati. Al contrario, entrambi i sistemi operano per l'intera finestra temporale disponibile. Il deficit prestazionale della variante con latenza non è dunque un problema di sopravvivenza, bensì di inefficienza logistica intrinseca: gli agenti con lag dispongono dello stesso tempo degli agenti baseline per eseguire consegne, ma ne completano significativamente meno, rivelando un'incapacità strutturale di pianificare e coordinare azioni efficaci in presenza di informazione ritardata. Questo distinguo è fondamentale per inquadrare correttamente la natura del problema: la latenza non provoca il collasso fisico del sistema, ma ne erode silenziosamente la capacità operativa dall'interno.

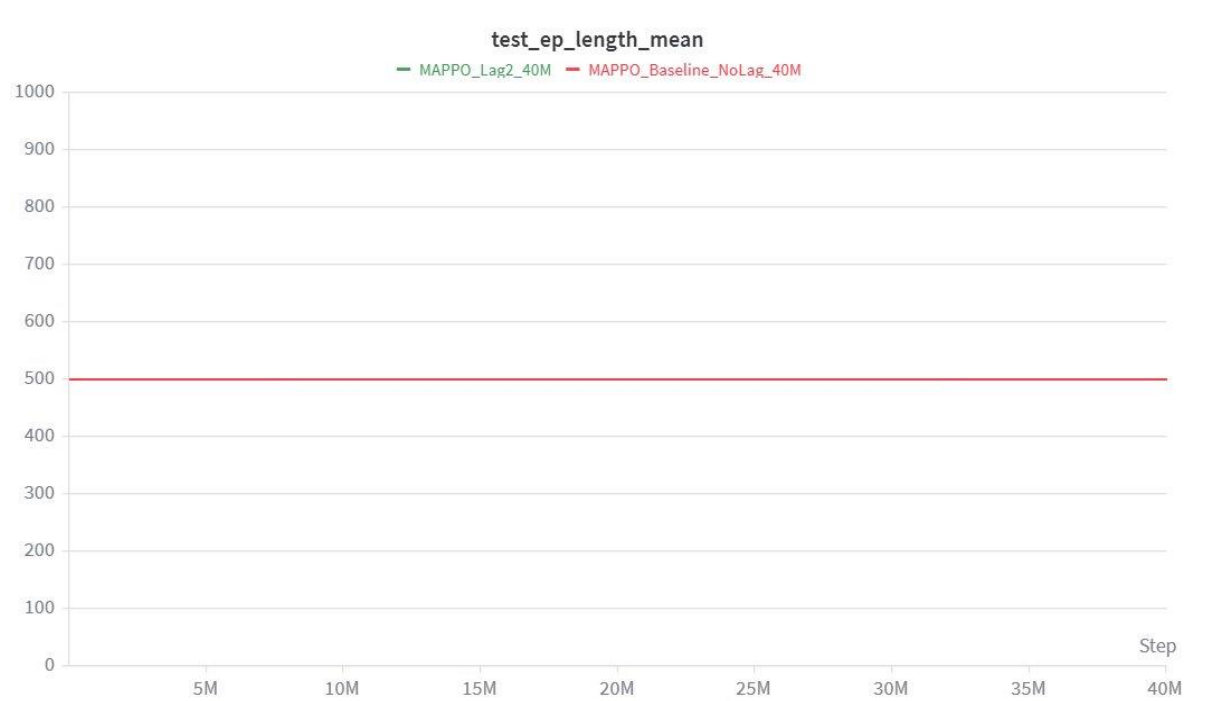


Figura 1 – test_ep_length_mean: la durata degli episodi è costantemente fissata a 500 step per entrambe le configurazioni per l'intera durata del training.

4.2 Impatto del Lag sulla Convergenza del Critico

L'analisi incrociata tra test_return_mean e critic_loss rivela il meccanismo causale centrale attraverso cui la latenza compromette l'apprendimento, e disegna due traiettorie evolutive profondamente asimmetriche.

La configurazione baseline MAPPO_Baseline_NoLag_40M esibisce una progressione coerente e ordinata: il test_return_mean (linea rossa) inizia a crescere in modo sostenuto intorno ai 13–14 milioni di step, raggiunge il plateau ottimale di circa 2.0 attorno ai 20–25 milioni di step, e vi si stabilizza con varianza molto ridotta fino al termine del training. In perfetta correlazione con questo andamento, la critic_loss della baseline si azzerava progressivamente: dopo una fase iniziale di alta variabilità, essa converge a valori prossimi allo zero entro i 20–22 milioni di step e vi rimane stabile. Questa co-evoluzione descrive un sistema in cui il Critico centralizzato acquisisce rapidamente una stima accurata della value function, fornendo segnali di addestramento affidabili agli attori e consentendo un apprendimento efficiente della policy.

La configurazione MAPPO_Lag2_40M mostra un quadro radicalmente differente e patologico. La critic_loss verde rimane ostinatamente elevata e altamente rumorosa per un intervallo di tempo enormemente più lungo — approssimativamente fino ai 28–30 milioni di step — con picchi che raggiungono valori superiori a 60, ampiamente superiori a quelli registrati dalla baseline nelle sue fasi di maggiore instabilità. Questa persistenza dell'errore del Critico ha una spiegazione diretta nel meccanismo della latenza: il Critico riceve osservazioni già obsolete al momento del loro utilizzo, poiché lo stato del mondo che esse descrivono appartiene a un passato di 2 step. La conseguenza è una cecità informativa sistematica: il Critico non riesce a costruire una mappa coerente tra stati e valori attesi, poiché lo stato osservato e lo stato effettivo in cui l'agente agisce sono strutturalmente disaccoppiati. Fino a quando questo errore rimane elevato, l'intero segnale di addestramento risulta degradato. Non sorprende, quindi, che il test_return_mean verde rimanga ancorato a valori bassi e mostri una crescita estremamente lenta fino a circa i 25–27 milioni di step, accumulando un ritardo di apprendimento di circa 10–12 milioni di step rispetto alla baseline.

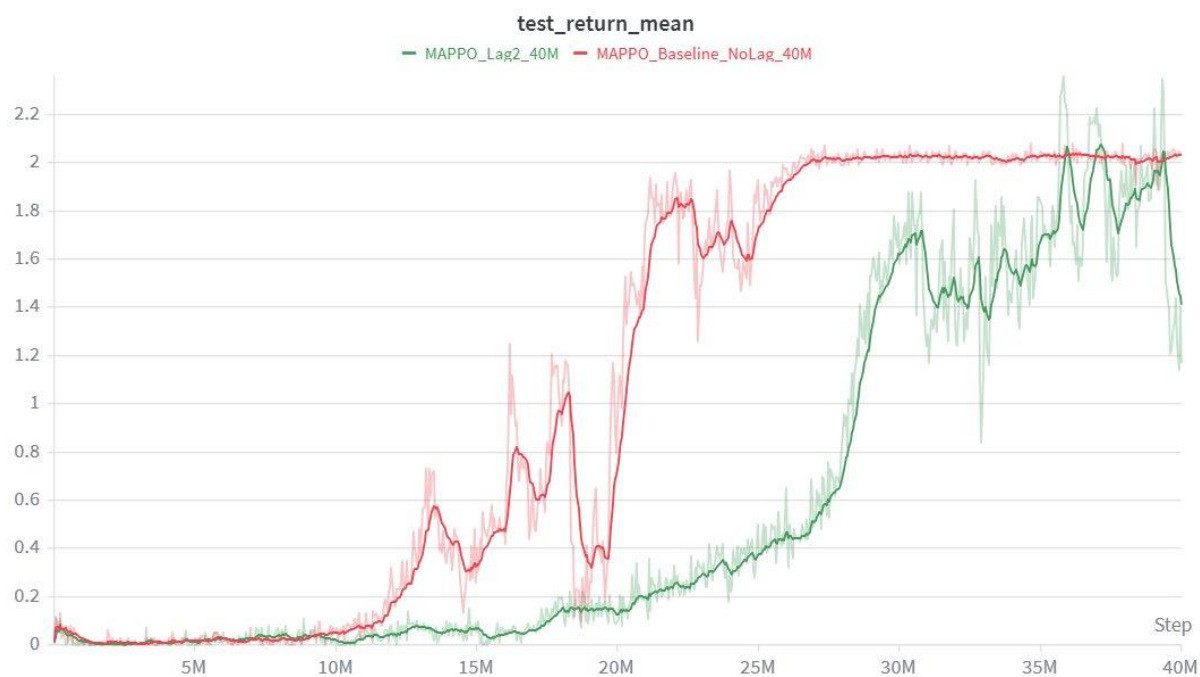


Figura 2 – `test_return_mean`: la baseline converge stabilmente entro i 25M step, mentre la variante con lag accumula un ritardo significativo di circa 10–12 milioni di step.

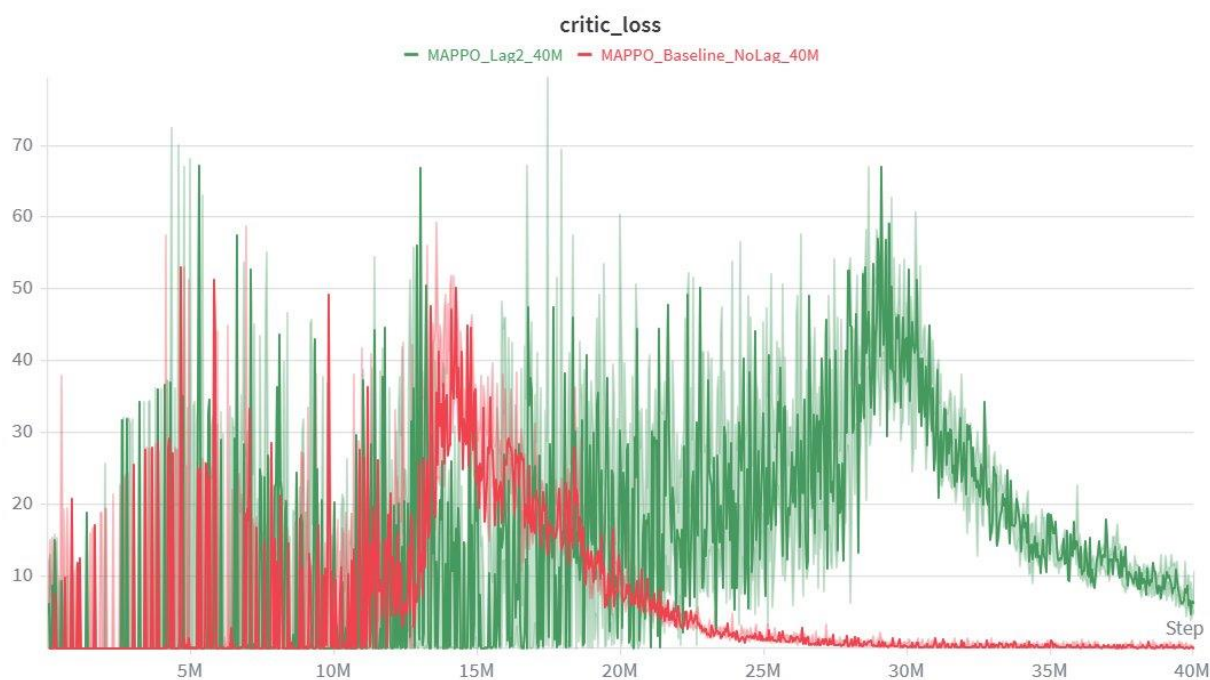


Figura 3 – `critic_loss`: la loss della baseline si azzerava rapidamente entro i 20–22M step, mentre quella della variante con lag rimane elevata e rumorosa fino a circa 30M step, rivelando la cecità informativa del Critico.

4.3 Instabilità, Shock ai Gradienti e Degrado delle Policy

Se l'analisi incrociata tra return e critic loss rivela il perché il lag rallenta l'apprendimento, l'osservazione congiunta di `test_return_mean` e `agent_grad_norm` espone la sua conseguenza più grave: la fragilità strutturale della policy appresa e la sua tendenza al collasso improvviso.

Nel corso dell'intero training, la norma dei gradienti degli agenti baseline rimane contenuta e controllata, con rare e modeste escursioni al di sopra della soglia di 2.5, a testimonianza di un processo di ottimizzazione stabile. La configurazione con lag presenta invece un profilo dei gradienti caratterizzato da picchi anomali di ampiezza sproporzionata. Il più vistoso di questi si verifica intorno ai 14–15 milioni di step, dove la norma del gradiente verde raggiunge un valore di circa 28–29, quasi venti volte superiore al valore di regime. Questo spike si colloca esattamente nel momento in cui il sistema con lag sta tentando la propria prima transizione verso performance più elevate, ma incontra gradienti così instabili da rendere l'ottimizzazione caotica e non direzionata.

Il caso più diagnosticamente rilevante è tuttavia quello che si verifica intorno ai 36–37 milioni di step. A quella profondità di training, la configurazione MAPPO_Lag2_40M aveva finalmente raggiunto il plateau prestazionale ottimale di circa 2.0 nel test_return_mean, apparentemente allineandosi con la baseline dopo un percorso di apprendimento molto più accidentato. L'agent_grad_norm mostra però, in corrispondenza esatta di quella finestra temporale, un nuovo picco anomalo di ampiezza significativa (circa 6–7, ben al di sopra della norma). La correlazione con ciò che accade nel test_return_mean è immediata e inequivocabile: subito dopo quel picco, il return della linea verde crolla bruscamente da circa 2.0 a valori intorno a 1.2–1.4, descrivendo un collasso catastrofico della policy che vanifica l'apprendimento accumulato fino a quel momento.

Questo evento dimostra con particolare chiarezza la natura della vulnerabilità introdotta dalla latenza. In un ambiente con osservazioni ritardate, il landscape di ottimizzazione che il sistema percepisce è sistematicamente distorto rispetto alla realtà dell'ambiente. Una policy che sembra aver convergito a un ottimo locale può trovarsi improvvisamente esposta a una regione dello spazio dei parametri in cui i gradienti sono instabili e gli aggiornamenti della policy producono effetti imprevedibili e controproducenti. La baseline, avendo appreso su osservazioni coerenti e tempestive, costruisce una policy robusta che non mostra vulnerabilità analoghe; il lag, al contrario, introduce una fragilità latente che può manifestarsi in qualsiasi momento, rendendo il sistema fondamentalmente inaffidabile anche quando le sue metriche aggregate sembrano indicare una convergenza soddisfacente.



Figura 4 – agent_grad_norm: i picchi anomali della variante con lag (in particolare a ~15M e ~37M step) sono direttamente correlati alle instabilità osservate nel test_return_mean, con il picco a 37M che causa un collasso catastrofico della policy.